

1vorlesung

November 12, 2025

```
[1]: from IPython.display import display, HTML

display(HTML('''
<style>
  /* Globale Stildefinitionen für RISE */
  /* Schriftgröße für den Code in Codezellen erhöhen */
  .reveal .input_area p {
    font-size: 3.5em !important; /* Ändert die Schriftgröße für den Code */
  }
  .reveal .output_area {
    font-size: 3.5em !important; /* Ändert die Schriftgröße für den Output_
↪*/
  }
  /* Optional: Padding und Margins anpassen */
  .reveal input_area, .reveal output_area {
    padding: 10px; /* Fügt mehr Innenabstand hinzu */
    margin: 10px; /* Fügt mehr Außenabstand hinzu */
  }
  .reveal p {
    font-size: 2em; /* Ändert die Schriftgröße für Fließtext */
  }
/* Globale Stildefinitionen für RISE */
  .reveal ptable {
    width: 100%; /* Ändert die Breite der Tabelle auf 100% */
    font-size: 2em !important; /* Ändert die Schriftgröße innerhalb der_
↪Tabelle */
  }
  .reveal th, .reveal td {
    padding: 10px; /* Fügt Innenabstand zu den Zellen hinzu */
    text-align: left; /* Ausrichtung des Textes in den Zellen */
  }
</style>
'''))
```

<IPython.core.display.HTML object>

1 Einführung in Datenauswertung mit Python

1.0.1 Vorlesung Theoretische Chemie (WiSe 2024/2025)

Jannis Kockläuner

jannis.kocklaeuner@tu-dresden.de

2 Motivation: Automatische Visualisierung

2.0.1 Beispiel: IR-Spektrum

2.0.2 Vorteile des Programmierens für Abbildungen

- **Reproduzierbar:** Code bleibt bestehen - der Aufwand weitere Abbildungen im selben Stil zu erzeugen beträgt praktisch 0.
- **Wiederverwendbar:** Kleine Änderung lassen sich leicht und schnell durch Anpassungen an bestehendem Code durchführen - Copy und Paste funktioniert oft.
- **Flexibel:** Grafiken können beliebig verändert und angepasst werden. Das beinhaltet das Erstellen von mehreren Grafiken, das Hinzufügen weiterer Objekte (Pfeile, Linien, Kreise, ...) und viel mehr.
- **Hohe Qualität:** Konventionen wie Schriftgröße, Achsenbeschriftung etc. können leicht automatisiert werden und ermöglichen es schnell sehr hochwertige Grafiken zu erstellen.

3 Motivation: Automatische Datenauswertung

3.0.1 Auswertung von vielen, ähnlichen Daten: Beispiel IR-Spektrum

3.0.2 Vorteile des Programmierens für Datenauswertung

- **Reproduzierbar:** Code bleibt bestehen - die Analyse bleibt nachvollziehbar und wiederholbar, auch wenn die Daten sich ändern.
- **Automatisierung:** Die selbe Analyse lässt sich mit nahezu beliebig vielen Daten durchführen - ohne nennenswerten zusätzlichen Arbeitsaufwand.
- **Auswertung:** Die Datenauswertung kann beliebig komplex sein - eine “general purpose” Programmiersprache kann so ziemlich jede Art von Algorithmus ausführen
- **Zeitaufwand:** Nach der Implementierung laufen Skripte gewöhnlich in **Sekunden** - selbst wenn sie hunderte von Dateien auswerten.

4 Motivation: Animationen

Animationen können sehr hilfreich sein um komplexe Prozesse abzubilden. Das Programmieren bietet den Vorteil das sowohl die Daten als auch die Animation selber in einem Schritt erzeugt werden können.

Diffusion Monte Carlo Simulation eines harmonischen Oszillators

Simulation der Bewegung der Erde um die Sonne

5 Motivation: Physikalisches Verständnis von Vorgängen

Physik und Chemie sind experimentelle Wissenschaften - oft ist es einfacher zu sehen was passiert als einfach nur eine Gleichung anzuschauen. Durch **Simulationen** kann man oft intuitiver verstehen was passiert.

Atomorbital des Wasserstoff Atoms

Quantenmechanisches Doppelmuldenpotential

6 Beispiel: Quantenmechanischer Tunneleffekt

6.0.1 1D - Doppelmuldenpotential:

Hamiltonian: $\hat{H} = \frac{\hat{p}^2}{2} + x^4 - x^2 - 0.05 * x$ - **Klassische Zeitentwicklung:** Teilchen fällt in eine Mulde wenn die kinetische Energie nicht reicht um die Barriere zu überwinden - **Quantenmechanische Zeitentwicklung:** Das Teilchen kann tunneln und hat in beiden Mulden eine nicht verschwindende Wahrscheinlichkeit

7 Warum Python?

7.0.1 Ranking der beliebtesten Programmiersprachen

Quelle: [Hier](#)

- **Vielseitigkeit:** Python kann für eine Vielzahl von Anwendungen verwendet werden, darunter Webentwicklung, Datenanalyse, maschinelles Lernen, wissenschaftliche Berechnungen, Systemskripting, Automatisierung und vieles mehr.
- **Wir beschränken uns auf eine spezielle Anwendung (Datenverarbeitung) und nutzen dafür nur wenige, grundlegende Funktionalitäten!** Notwendigerweise werden dadurch viele Themen nur oberflächlich und/oder unvollständig behandelt.

8 Nützliche Ressourcen

8.0.1 Online Ressourcen

- **Google**
- **Stack Overflow**
- **Python Dokumentation**
- **Youtube Tutorials**

8.0.2 Coding Hilfen

- **Syntax Highlighting** - Findet Syntax Fehler im Code und ist Standard bei IDE's wie Pycharm
- **AI Assistenten** - ChatGPT & Co können guten Code schreiben - wenn **genau beschrieben** wird was gebraucht wird

9 Warum Python?

- **Einfach und leicht zu lesen:** Der Code lässt sich oft intuitiv verstehen, was die Sprache ideal für Einsteiger macht.
- Funktionen und Operatoren orientieren sich an englischen Begriffen
- Python verwendet Einrückungen um Codeblöcke zu kennzeichnen. Das erleichtert die Strukturierung und Lesbarkeit.

```
[ ]: # Beispiel: Lesbarkeit
my_number = 9
if my_number >= 10:
    print(my_number)
else:
    print("Die Zahl ist kleiner als 10!")
```

10 Warum Python?

- **Interaktiv und interpretierbar:** Python ist eine interpretierte Sprache, was bedeutet, dass der Code direkt Zeile für Zeile ausgeführt wird. Das macht es einfach, Code direkt auszuprobieren und zu testen.
- Der Code wird von einem Python Interpreter von oben nach unten gelesen und ausgeführt

Quelle: [Hier](#)

11 Fehlermeldungen: Computer sind dumm, aber Fehlermeldungen sind nützlich!

Wenn der Code nicht lesbar für den Compiler (Python) ist, wird eine Fehlermeldung produziert. **Fehlermeldungen in Python sind (meistens) sehr nützlich!** Sie enthalten wichtige Informationen um den Fehler zu finden und zu verbessern: - Fehlercode (NameError, IndexError, SyntaxError, ...) - Wo tritt der Fehler auf? - Um was für eine Art von Fehler handelt es sich konkret? Fehlermeldungen sind essentiell um Fehlfunktionen des Programms zu verhindern - ein Programm sollte nicht funktionieren wenn nicht eindeutig ist was es tun soll.

12 Warum Python?

- **Module und Bibliotheken:** Python verfügt über eine Vielzahl von Bibliotheken und Frameworks, die die Entwicklung erheblich erleichtern. Beispiele sind:
 - NumPy, Pandas, Matplotlib für Datenwissenschaft und -analyse
 - Django, Flask für Webentwicklung
 - TensorFlow, PyTorch für maschinelles Lernen
- Für komplexe Aufgaben ist es oft **einfacher und sinnvoller** nach dem passenden Paket zu suchen anstatt selbst sämtliche Funktionalitäten zu programmieren.

Quelle: [Hier](#)

13 Module und Bibliotheken: Beispiel Sinusfunktion

Definition der Sinusfunktion: $\sin(x) \approx x - \frac{x^3}{3!} + \frac{x^5}{5!} - \dots$

```
[ ]: # Beispiel: Sinusfunktion
def sinus(x, n):
    # Taylornäherung der Sinusfunktion
    # sin(x) = x - x^3/3! + x^5/5! - ...
    # x: Punkt der Sinusfunktion
    # n: Anzahl an Termen der Taylornäherung
    y = x # Output
    fac = 1 # Factorial
    for i in range(1, n):
        n_term = x ** (2 * i + 1)
        fac *= 2 * i * (2 * i + 1)
        sign = (-1) ** i
        y += sign * n_term / fac
    return y

x = 0.5
n = 3 # Länge der Taylorentwicklung
sin_x = sinus(x, n)
print(sin_x)
```

```
[ ]: # Ziel: Berechne die Sinusfunktion mit NumPy
import numpy as np # Importiere das NumPy (Numerisches Python) Paket
x = 0.5
sin_x = np.sin(x)
print(sin_x)
```

NumPy ist eine der wichtigsten Module für Python und wird in der nächsten Vorlesung ausführlicher vorgestellt.

14 Basics: Was ist ein Programm?

Ein Programm besteht aus einer Reihe von Anweisungen, die **nacheinander** ausgeführt werden. In einem Programm werden verschiedene Objekte, z.B. Zahlen, Listen oder Text, durch Operationen manipuliert, um ein gewünschtes Ergebnis zu erhalten.

14.0.1 Beispiel: Summiere die Zahlen von 1 bis 10

```
[ ]: # Beispiel: Programm
n = 10
my_number = 0
for i in range(1, n + 1):
    my_number = my_number + i

print(my_number)
```

Ein Python Programm ist eine **Textdatei**, die üblicherweise mit **.py** endet und Befehle in der Python Sprache enthält. Die Textdatei wird zu einem Python Programm, wenn sie durch einen **Python Interpreter** gelesen wird - üblicherweise durch einen Befehl in der **Kommandozeile**.

```
$ python my_script.py
```

Alternativ kann ein IDE (Integrated Development Environment) wie **PyCharm** genutzt werden, die sowohl das Programmieren in der Textdatei als auch die Ausführung des Programms in einer Entwicklungsumgebung vereint. Zusätzlich bieten Entwicklungsumgebungen nützliche Hilfestellungen, z.B. durch Syntax-Highlighting und Fehlererkennungen.

15 Einfachheit > Performance

Die Grundlage des Programmierens ist der Programmcode der von Entwickler*innen geschrieben wird. Guter Code folgt einfachen Prinzipien: - **Lesbarkeit** - Ordentlicher Code nutzt **Konventionen** für Variablen, Einrückungen, Kommentare - **Verständlichkeit** - Verständlicher Code ist **kommentiert** und **strukturiert** - **Konfigurierbarkeit** - Kleine Änderungen an der Aufgabe sollten zu kleinen Änderungen am Code führen - **Don't repeat yourself** - Wiederkehrende Aufgaben sollten zusammengefasst und automatisiert werden

15.0.1 Beispiel Lesbarkeit & Verständlichkeit: Fläche eines Kreises

$$A = \pi * r^2$$

<h2>Bad practice</h2>

```
[ ]: # Beispiel: Lesbarkeit
print(f"{3.1415 * (2 ** 2)}")
print(f"{3.1415 * (4 ** 2)}")
```

<h2>Good practice</h2>

```
[ ]: # Aufgabe: Berechne die Fläche eines Kreises A = pi * r^2
pi = 3.1415          # Kreiszahl Pi
r_1 = 2              # Radius Kreis 1
r_2 = 4              # Radius Kreis 2
A_1 = pi * (r_1 ** 2) # Fläche Kreis 1
A_2 = pi * (r_2 ** 2) # Fläche Kreis 2
# Textausgabe
print(f"Die Fläche von Kreis 1 beträgt ", A_1)
print(f"Die Fläche von Kreis 2 beträgt ", A_2)
```

Das Ergebnis ist zwar das selbe, aber die zweite Version ist **klarer und verständlicher** durch: - Kommentare # mit kurzen Beschreibungen/Definitionen - Sinnvolle Variablennamen - Informative Ausgabe der Ergebnisse

16 Übung 0: "Hello World"

Die **print()** Funktion ist **eine der wichtigsten Funktionen in Python**: Sie gibt Daten für uns lesbar aus - nützlich zum **debuggen** und um **Ergebnisse** zu erhalten. **### Syntax:**

```
print(*Daten_zum_ausgeben*)
```

```
[ ]: print("Hello World")
```

17 Übung 0.5: Wann nutzt man print()?

Oft ist es nicht offensichtlich, wie, wo und welche Daten genau an einer Stelle des Codes gespeichert werden. Die `print()` Funktion kann **Zwischenergebnisse** sichtbar machen. `###` Aufgabe: Gib das Resultat für "variable3" aus

```
[ ]: # Definiere eine Variable
variable1 = "Hello "
variable2 = "World"
variable3 = 3 * variable1 + variable2
# Unsicher was variable3 enthält? Guck es dir an!
```

18 Übung 1: Debuggen mit Fehlermeldungen

Um zu funktionieren, folgen Programmiersprachen festen Regeln. Machen wir bei der Einhaltung der Regeln Fehler, bricht das Programm (hoffentlich) ab! **Python hilft euch Fehler zu finden** durch Fehlermeldungen. Fehlermeldungen beinhalten die folgenden Informationen: - In welcher Zeile tritt der Fehler auf? - An welcher Stelle tritt der Fehler auf? - Was ist das Problem? `###` Aufgabe: Finde alle Fehler durch die Fehlermeldungen

```
[ ]: pi = 3.1415          # Kreiszahl Pi
r_1 =                    # Radius Kreis 1
r_2 = 4                  # Radius Kreis 2
A_1 = pi * (r_1 ** 2)    # Fläche Kreis 1
A_2 = pi * (r_2 ** 2)    # Fläche Kreis 2
# Textausgabe
print(f"Die Fläche von Kreis 1 beträgt , A_1)
print(f"Die Fläche von Kreis 2 beträgt ", A2)
```

18.0.1 Google Colab Link

Colab Link: <https://colab.research.google.com/drive/1CqGHfmt7z8T2HMTtNQVOnWrbzN3OBXAr?usp=sharing>

19 Basics: Variablen

Variablen bilden die Grundlage eines Programms und dienen der **Datenspeicherung**. Eine Variable ist definiert durch drei Dinge:

- **Bezeichner:** Der Name der Variable, z.B. `my_number`
- **Datentyp:** Die Art der Daten die in der Variable gespeichert werden, bspw. Zahlen, Text, eine Liste von Zahlen oder ein Plot
- **Wert:** Der spezifische Wert der Variable, bspw. eine spezifische Zahl oder eine Grafik

Die **Bezeichner** einer Variable können frei gewählt werden, innerhalb der von Python gesetzten Regeln für Variablen: - **Python ist case-sensitive**: Groß und Kleinschreibung (`omega`, `Omega`, ...) spielt bei der Benennung von Variablen eine Rolle - **Erlaubte symbole**: `a ... z`, `0 ... 9` und `_` (**nicht**: `-`) - **Geschützte Namen**: Namen von speziellen Funktionen und Befehlen sind geschützt und können nicht überschrieben werden

```
[ ]: # Bezeichner = Wert
my_variable = 3
```

<h3>Wichtig</h3>

<p>Variablen sollten deskriptiv sein, d.h. der Name einer Variable sollte darauf schließen lassen

Mehrere Variablen können durch ein Komma `,` getrennt werden, sowohl beim definieren als auch beim ausgeben

```
[ ]: # Definition mehrerer Variablen
number1, number2 = 1, 2
print(number2, number1)
```

20 Datentypen: Zahlen

Es gibt zwei unterschiedliche Arten von Zahlen: Ganze Zahlen (**integers**) oder Gleitkommazahlen (**floats**).

```
[ ]: # integer (Ganzzahl)
a = 1
b = -5
c = 0
print(a, b, c)

d = 0.1 + 0.2
e = 0.3
f = d - e
print(d, e, f)
```

Floats können auch in wissenschaftlicher Notation mit einem Exponenten als $1000 = 10e3$ und $0.001 = 10e-3$ angegeben werden. Während **Integers** in Python eine beliebige Größe haben können sind **Floats** auf Zahlen zwischen $1.7 * 10^{-308}$ bis $1.7 * 10^{308}$ begrenzt, da jede Zahl lediglich 64 Bits Speicher zur Verfügung steht. Höhere Zahlen sind nicht speicherbar.

```
[ ]: # Overflow
a = 1e307
b = 2e308
print(a, b)
```

Floats beschreiben nur mit einer begrenzten Genauigkeit das Kontinuum der reellen Zahlen, welche als **machine precision** bezeichnet wird.

```
[ ]: a = 0.1 + 0.2
      b = 0.3
      print(b, f"{b:.20f}")
      print(b - a)
```

<h3>Wichtig</h3>

<p>In Python werden Floats mit einem Punkt <code>.</code> geschrieben, nicht mit einem Komma w

21 Basics: Operationen mit Zahlen

Variablen die **Integers** oder **Floats** enthalten, können mit grundlegenden mathematischen Operatoren genutzt werden.

Operation	Symbol	Beschreibung
Addition	+	Addiert zwei Werte
Subtraktion	-	Subtrahiert den zweiten Wert vom ersten
Multiplikation	*	Multipliziert zwei Werte
Division	/	Dividiert den ersten Wert durch den zweiten
Modulo (Rest)	%	Gibt den Rest einer Division zurück
Potenz	**	Potenziert den ersten Wert mit dem zweiten
Quadratwurzel	** 0.5	Zieht die Quadratwurzel des Wertes

```
[ ]: a = 2.0
      b = 3

      # Addition:
      add = a + b
      # Subtraktion:
      sub = a - b
      # Multiplikation
      mul = a * b
      # Division
      div = a / b
      # Modulo
      mod = a % b
      # Exponent
      power = a ** b

      print(add, sub, mul, div, mod, power)
```

Die Reihenfolge der mathematischen Operationen folgt der Standardreihenfolge mathematischer Operationen (BODMAS): **Klammern, Exponenten, Division, Multiplikation, Addition, Subtraktion** - Kurz gesagt: Punkt vor Strich

```
[ ]: # Klammern vor Exponenten, Exponenten vor Division
      a = 9 ** 1 / 2
      b = 9 ** (1 / 2)
```

```
print(a, b)
```

```
[ ]:
```

Klammern setzen ist sehr hilfreich um schwer zu entdeckende Fehler zu vermeiden. Sämtliche Klammern müssen allerdings geschlossen sein, da ansonsten ein Fehler produziert wird.

22 Operationen mit Zahlen

Zusätzlich zu den grundlegenden Operatoren gibt es eine Vielzahl an Funktionen, die auf Zahlen angewandt werden können.

Operation	Symbol	Beschreibung
Absolutwert	<code>abs()</code>	Gibt den Betrag einer Zahl zurück
Rundung	<code>round()</code>	Rundet eine Zahl auf eine bestimmte Dezimalstelle
Maximum	<code>max()</code>	Gibt den größten Wert aus einer Sequenz zurück
Minimum	<code>min()</code>	Gibt den kleinsten Wert aus einer Sequenz zurück

```
[ ]: a = -2.21
      b = 3

      # Absolutwert
      abs_val = abs(a)
      # Rundung
      round_val = round(a, 1)
      # Maximalwert
      max_val = max(a, b)
      # Minimalwert
      min_val = min(a, b)

      # Ausgabe
      print(abs_val, round_val, max_val, min_val)
```

Spezielle mathematische Funktionen, wie bspw. die Exponentialfunktion, werden in Python durch Module bereitgestellt.

23 Übung 2: Rechenoperationen

23.0.1 Aufgabe: Verändere den Code, sodass `my_integer` durch die Subtraktion 6 enthält. Lasse dir das Ergebnis mit `print()` ausgeben.

```
[ ]: my_integer = 9
      my_integer - 3
      print(my_integer)
```

23.0.2 Aufgabe: Wende auf `my_integer` eine Addition von 2, Multiplikation mit 3, Subtraktion von 1 und abschließend eine Division durch 2 an.

```
[ ]: my_integer = 9
my_integer = my_integer # Addition
my_integer = my_integer # Multiplikation
my_integer = my_integer # Subtraktion
my_integer = my_integer # Division
print()
```

23.0.3 Aufgabe: Finde das Maximum von `Wurzel(3333)` und `331.5`

```
[ ]: sqrt = # Wurzel von 3333
expo = # Exponential 331.5
max_val = # Find Maximum
# Print out maximum
```

23.0.4 Aufgabe: Finde mit dem modulo Operator heraus ob `74` eine gerade Zahl ist

```
[ ]: my_number = 7 # 74
mod = my_number #

#####
# Nicht verändern
#####
if mod != 0:
    print(f"{my_number} ist eine ungerade Zahl")
else:
    print(print(f"{my_number} ist eine gerade Zahl"))
```

24 Datentypen: Container

Variablen können zusammengefasst werden in Containern, die Operationen auf Gruppen von Daten erlauben. Es gibt 4 verschiedene grundlegende Arten von Containern:

- **Listen:** Listen enthalten geordnete Daten, die überschrieben werden können
- **Tupel:** Tupels enthalten geordnete Daten, die **nicht** überschrieben werden können
- **Sets:** Sets enthalten **ungeordnete, einzigartige** Daten
- **Dictionaries:** Dictionaries enthalten ungeordnete Key-Value Paare, bei denen jedem Key ein Objekt zugeordnet ist

Für den Moment konzentrieren wir uns auf **Listen**. Eine **Liste** enthält eine Menge an einzelnen Elementen (**Integers, Floats, Strings, Bools**), die an einer bestimmten Position, einem Index, gespeichert sind. Der Index zählt die Elemente in der Liste, beginnend bei 0. Auf einzelne Elemente der Liste kann über den Index zugegriffen werden mittels eckiger Klammern `liste[index]`. Wenn versucht wird auf ein Element zuzugreifen, dass nicht in dem Container enthalten ist, wird ein Fehler produziert. Auf mehrere Elemente kann durch **Slicing** `list[index1:index2]` zugegriffen werden, wodurch alle Elemente zwischen den Indices ausgegeben werden.

```
<h3>Wichtig</h3>
<p>
    In Python wird immer ab 0 gezählt.
</p>
```

```
[ ]: # Listen Basics
elements = ["Hydrogen", "Helium", "Lithium", "Berrylium", "Boron", "Carbon",
            ↪ "Nitrogen"]
print(elements[0], elements[-1])
print(elements[0:4])
```

Quelle: [Hier](#)

25 Listen: Operationen

Listen können mit einer Vielzahl an Funktionen manipuliert werden.

Operation	Beschreibung	Syntax
Hinzufügen	Fügt ein Element am Ende der Liste hinzu	<code>liste.append(element)</code>
Einfügen	Fügt ein Element an einem bestimmten Index ein	<code>liste.insert(index, element)</code>
Löschen	Löscht das Element am angegebenen Index	<code>del liste[index]</code>
Länge bestimmen	Gibt die Anzahl der Elemente in der Liste zurück	<code>len(liste)</code>
Sortieren	Sortiert die Liste in aufsteigender Reihenfolge	<code>liste.sort()</code>
Zählen	Zählt, wie oft ein Element in der Liste vorkommt	<code>liste.count(element)</code>
Finden	Gibt den Index des ersten Vorkommens eines Elements	<code>liste.index(element)</code>

```
[ ]: elements = ["Hydrogen", "Helium", "Lithium", "Berrylium", "Boron", "Carbon",
            ↪ "Nitrogen"]
elements.append("Oxygen")
print(elements)
elements.index("Carbon")
```

Listen sind keine Vektoren und sind, genau wie **Strings**, im Allgemeinen nicht kompatibel mit mathematischen Operatoren.

```
[ ]: my_list = [1, 2, 3, 4]
my_list = my_list * my_list
print(my_list)
```

26 Datentypen: Strings

Text, oder **Strings**, wird in Python durch Anführungszeichen `' '` oder `" "` markiert, um von **Variablen** unterschieden zu werden.

```
[ ]: my_text = "Hello World"
      print(my_text)
```

Strings können als Abfolge einzelner Zeichen, **Characters** betrachtet werden, die also genau wie bei Listen mittels eckiger Klammern separat aufgerufen werden können.

```
[ ]: my_text = "Hello World"
      print(my_text[0], my_text[1], my_text[2], my_text[3], my_text[4])
```

Strings sind ein fundamental anderer Datentyp als **Floats** und **Integers** und können nur in Ausnahmefällen kombiniert werden.

```
[ ]: my_text = "10"
      my_int = 2
      print(my_text + my_int)
```

Strings die Zahlen enthalten müssen erst in Floats oder Integers umgewandelt werden bevor sie als solche genutzt werden können.

```
[ ]: my_text = "10.1"
      my_int = 2
      my_text = float(my_text)
      print(my_text + my_int)
```

Strings können mit einer Vielzahl an Operationen manipuliert werden

Operation	Beschreibung	Beispiel	Ausgabe
Verkettung	Verbindet Strings mit +	"Hallo, " + "Welt"	Hallo, Welt
Länge bestimmen	Anzahl der Zeichen mit len()	len("Python")	6
Ersetzen	Ersetzt Substrings mit replace()	"Ich liebe Python".replace("Python", "Coding")	Ich liebe Coding
Aufteilen	Teilt String in eine Liste von Wörtern	"eins zwei drei".split()	['eins', 'zwei', 'drei']

Variablen können in Text mittels der `format()` Funktion oder als `f" "` **String** eingefügt werden.

```
[ ]: # Gibt eine berechnete Konzentration aus
      concentration = 0.2 # mol / l
      # f" " string method
      print(f"Die finale Konzentration beträgt {concentration} mol / l")
      # format function
```

```
print("Die finale Konzentration beträgt {} mol / l".format(concentration))
```

27 Übung 3: Operationen mit Listen und Strings

27.0.1 Aufgabe: Berechne die molare Masse von Glucose mithilfe der Liste der molaren Massen und des Strings der Summenformel.

```
[ ]: my_list = [12, 1, 16]    # molare Massen: C, H, O
     my_string = "C6H12O6"  # Glucose

     kohlenstoff = int(my_string[1]) * my_list[0]
     wasserstoff =
     sauerstoff =
     mol_mass = # Addiere die einzelnen Massen
     print(f"Die molare Masse von Glucose beträgt {mol_mass} g/mol.") # Wende den
     ↪ f"-String an um die molare Masse ausgeben zu lassen.
```

27.0.2 Aufgabe: Berechne die Konzentration aller Stoffe

Hinweise: - Durch Slicing kann man sich bspw. mit `list[::-2]` jeden zweiten Wert einer Liste ausgeben lassen - Strings können durch die `.split()` Funktion in Listen überführt werden - In einer Liste können strings durch die `map` Funktion in floats oder integers umgewandelt werden: `list(map(int, my_list))` oder `list(map(float, my_list))`

```
[ ]: # Struktur: Stoff, Masse [g], Molmasse [g/mol], Volumen Lsgmittel [mL]
     c_string = "C6H6, 1, 78, 100, C2H4O2, 0.8, 60, 95, C5H5N, 1.1, 79, 110"
     c_list = c_string.split(",")

     mol = c_list[::4]
     mass = c_list[1::4]
     mass = list(map(float, mass))
     mol_mass = c_list[2::4]
     mol_mass = list(map(float, mol_mass))
     vol = c_list[3::4]
     vol = list(map(float, vol))

     c1 = (mass[0] / mol_mass[0]) / vol[0] * 1000
     c2 = (mass[1] / mol_mass[1]) / vol[1] * 1000
     c3 = (mass[2] / mol_mass[2]) / vol[2] * 1000

     print(f"Die Konzentration von {mol[0]} beträgt {c1} mol/L.")
     print(f"Die Konzentration von {mol[1]} beträgt {c2} mol/L.")
     print(f"Die Konzentration von {mol[2]} beträgt {c3} mol/L.")
```

28 Datentypen: Bool Werte

Bool'sche Werte können nur zwei Werte annehmen: Wahr oder Falsch, bzw. in Python **True** und **False**. Sie spielen eine wichtige Rolle in der Strukturierung von Programmen, bspw. indem gewisse Operationen nur durchgeführt werden wenn ein Wert **True** ist.

Oft werden **Bool**'sche Werte durch **Vergleichsoperatoren** oder **logische Operationen** erzeugt. **Vergleichsoperatoren** testen eine Bedingung zwischen zwei einzelnen Datenobjekten, bspw. zwischen zwei Zahlen oder zwei **Strings**, und geben einen **Bool**'schen Wert zurück.

```
[ ]: # Vergleichsoperatoren
a = 1
b = 2

# Kleiner
lower = a < b
# Größer
greater = a > b
# Gleich
eq = a == b
# Ungleich
neq = a != b
# Größer Gleich
greatereq = a >= b
# Kleiner Gleich
lowereq = a <= b

print(lower, greater, eq, neq, greatereq, lowereq)
```

<h3>Wichtig</h3>

<p>

Vergleiche zwischen <code>Floats</code> sind oft schwierig, da durch die begrenzte Maschinerie

</p>

```
[ ]: a = 0.1 + 0.2
b = 0.3
eq = a == b

print(eq)
```

29 Logische Operatoren

30 Logische Operatoren

Logische operatoren kombinieren zwei **Boolsche** Werte und geben gemäß ihrer Logik eine neue **Bool** zurück. | **Operator** | **Beschreibung** | **Beispiel** | **Ergebnis** |

-----|-----|-----|-----|

and | Gibt True zurück, wenn beide Operanden True sind. | True and True | True | | | True and False | False | | or | Gibt

True zurück, wenn mindestens einer der Operanden True ist. | True or False | True ||| False
or False | False || not | Negiert den Booleschen Wert des Operanden. | not True | False |||
not False | True |

```
[ ]: # Logische Operatoren
a = True
b = False
# Und
und = a and b
# Oder
oder = a or b
# XOR
entweder_oder = a ^ b
# Nicht
nicht = not a

print(und, oder, entweder_oder, nicht)
```

Bool'sche Werte können für mathematische Operationen genutzt werden, wobei sie entweder die Werte True = 1 oder False = 0 annehmen.

```
[ ]: a = True
b = 2.7
mult = a * b
add = a + b
div = a / b

print(mult, add, div)
```

Umgekehrt können logische Operationen zwischen sämtlichen Objekten durchgeführt werden. Dabei sind sämtliche einzelnen Datenobjekte True. Die einzige Ausnahme bildet die **Integer** 0, die False ist. Variablen, die zwar deklariert wurden aber keinen Wert haben, bspw. ein leerer **String** "", sind ebenfalls False.

```
[ ]: a = "Apfel"
b = 0

und = a and b
oder = a or b
nicht = not a

print(und, oder, nicht)
```

31 Übung 4: Logische Operationen

31.0.1 Aufgabe: Wende logische Operatoren sowie Vergleichsoperatoren an um mit den Variablen einen sinnvollen Output aus den gegebenen Beispielen zu erzeugen.

```
[ ]: # Beispiel:
Vorlesungsbeginn = 7.5 # Uhrzeit
Wecker_klingelt = 8.0
# Logische Schlussfolgerung:
print("Schaffe ich es zur Vorlesung?", Wecker_klingelt < Vorlesungsbeginn)

Siedepkt_Wasser = 100
Temperatur1 = 80
print(f"Damit es siedet muss Wasser der Temperatur {Temperatur1} °C weiter_
    erhitzt werden:", )

Masse_Glucose = 180
Masse_Ribose = 150
print("Glucose ist ein Isomer von Ribose:", )

Hitze = True
Katalysator = False
print("H2O2 zersetzt sich:", )

Kittel = True
Schutzbrille = False
print("Studierende/r darf das Labor betreten:", )

Glasgerät_dreckig = True
print("Das Glasgerät kann verwendet werden:", )
```

32 If/Else statements: Entscheidungen

Quelle: [Hier](#)

33 If/Else statements: Entscheidungen

In Python gibt es Entscheidungsstrukturen, sogenannte **If/Else** Blöcke. Sie folgen der Syntax **if Bedingung: ... else:** Dabei wird der erste Code Block ausgeführt wenn die Bedingung erfüllt ist, während der andere ausgeführt wird wenn sie **nicht** erfüllt ist. Die Code-Blöcke durch ihre Einrückung voneinander getrennt.

```
[ ]: # If-else
color = "rot"

if color == "rot":
```

```

    print("Lithium!")
else:
    print("Natrium!")

```

34 If - Elif - Else

Eine Bedingung kann durch einen **Bool** Wert ersetzt werden oder mehrere Bedingungen verknüpfen gemäß der **Bool** Logik. Mehrere Spezialfälle können mit durch das **elif** Keyword hinzugefügt werden.

```

[ ]: # If-elif-else
color = "blau"

if color == "rot":
    print("Lithium!")
elif color == "gelb":
    print("Natrium!")
else:
    print("Kalium!")

```

35 Kontrollstrukturen

Quelle: [Hier](#)

36 Kontrollstrukturen: Iterieren & Loops

Anstatt einzeln auf die Elemente einer Liste zuzugreifen, kann eine Operation nacheinander auf alle Elemente einer Liste angewandt werden in einem sogenannten **for** loop. Dabei wird die Liste Index für Index durchgegangen (**iteriert**), bis alle Elemente einmal behandelt wurden. Der Loop wird eingeleitet durch das **Keyword** **for ... in** und folgt der allgemeinen Syntax **for [name] in [iterable]:**

```

[ ]: my_list = [5, 3, 6, 2]
for element in my_list:
    print(element)

```

Quelle: [Hier](#)

<h3>Wichtig</h3>

<p>

Ein For-Loop ist eine Kontrollstruktur, innerhalb derer der darauffolgende Code

37 For-Loops

Loop	Funktion	Syntax
Liste	Loop über Elemente einer einzelnen Liste	for el in list:

Loop	Funktion	Syntax
Zahlen	Loop über Zahlen von <code>start</code> bis <code>end</code>	<code>for element in range(start, end):</code>
mehrere Listen	Loop über die Elemente mehrerer Listen	<code>for el1, el2, ... in zip(list1, list2, ...):</code>
Index und Liste	Loop über den Index und die Elemente einer Liste	<code>for idx, el in enumerate(list):</code>

Ein For-Loop muss nicht direkt über die Elemente einer Liste iterieren. Oft ist es leichter, über Zahlen, bspw. die Indices einer Liste, zu loopen. Dies wird mit der `range()` Funktion erreicht.

```
[ ]: my_list = [5, 3, 6, 2]
      n = len(my_list)

      for i in range(n):
          my_list[i] = my_list[i] + 1

      print(my_list)
```

Der Code innerhalb einer Kontrollstruktur kann beliebig lang und komplex sein, und es ist auch möglich (und häufig) mehrere Loops ineinander zu schachteln.

Eine kompaktische und praktische Methode um kurze Operationen auf Listen auszuführen sind **List Comprehensions**. Diese folgen einer einfachen einzeiligen Syntax:

```
[ ]: squares = [x ** 2 for x in range(10)]
      print(squares)
```

38 Übung 5: For-loops

38.0.1 Aufgabe: Rechne die Temperaturen in der Liste mithilfe eines for-loop in °C um.

```
[ ]: temperaturen = [235, 342, 600, 273] # Temperaturen in Kelvin

      for :

      print(temperaturen)
```

38.0.2 Aufgabe: Rechne die Temperaturen in der Liste mithilfe einer List comprehension in °C um.

```
[ ]: temperaturen = [235, 342, 600, 273] # Temperaturen in Kelvin

      temperaturen = # List comprehension

      print(temperaturen)
```

38.0.3 Aufgabe: Entferne alle Elemente der Liste die fehlerhaft sind

Hinweise: - Nutze if/else statements - Mit List comprehension kann die Aufgabe in zwei Zeilen gelöst werden

```
[ ]: wavenumber = [1000, 1010, 1020, 1030, 1040, 1050, 1060, 1070, 1080, 1090, 1100, 1110, 1120, 1130, 1140, 1150, 1160, 1170, 1180, 1190, 1200]
      intensity = [0.000, False, 0.045, 0.052, 0.030, 0.020, False, 0.75, 0.082, 0.083, 0.069, 0.038, 0.010, 0.004, 0.030, 0.077, 0.091, 0.059, 0.090, 0.072, False]
```

39 Loops über mehrere Variablen

Anstatt nur eine einzelne Variable in einem **For-Loop** zu nutzen, können mit der `zip` Funktion auch mehrere Variablen in einem einzelnen Loop genutzt werden wenn sie die gleiche Länge haben.

```
[ ]: molecules = ["Mol1", "Mol2", "Mol3"]
      weights = [109.3, 165.9, 93.5]
      for mol, w in zip(molecules, weights):
          print(f"{mol} wiegt {w} g / mol")
```

Um gleichzeitig über den Index und ein Element aus einem iterierbaren Objekt zu loopen, kann die `enumerate()` Funktion genutzt werden.

```
[ ]: molecules = ["Mol1", "Mol2", "Mol3"]
      weights = [109.3, 165.9, 93.5]
      for i, mol in enumerate(molecules):
          w = weights[i]
          print(f"{i}. {mol} wiegt {w} g / mol")
```

40 Übung 6: Loops über mehrere Variablen und if-else statements

40.0.1 Aufgabe: Es sind in der Liste “temperaturen” fünf Schmelzpunkte für verschiedene Elemente in “stoffe” gelistet. Die Listen sind bereits sortiert, sodass z.B. der erste Schmelzpunkt zum ersten Element in “stoffe” gehört. Lass mithilfe einer loop für die Einträge der Liste den Schmelzpunkt mit dem jeweiligen Element als Aufzählung ausgeben. Also beispielhaft nach dem Schema:

1. Der Schmelzpunkt von **Wasserstoff** liegt bei **-259 °C**.
2. Der ...

Einer der Werte ist allerdings unphysikalisch und sollte zuvor mithilfe einer List comprehension entfernt werden.

```
[ ]: temperaturen = [14, 303, -10, 505, 337] # Schmelzpunkte in Kelvin
      stoffe = ["Wasserstoff", "Gallium", "Helium", "Zinn", "Kalium"] # zugehörige Stoffe
      temp_masked = # auszufüllen
```

```

stoffe_masked = # auszufüllen

for in :

    print(f"")

```

40.0.2 Aufgabe: Entferne alle unnötigen Wiederholungen und gib nur den Stoff mit der höchsten Konzentration aus.

Hinweise: - Nutze "List comprehension" anstelle von map Funktionen - Mit zip() kann man über mehrere Listen gleichzeitig loopen

```

[ ]: # Struktur: Stoff, Masse [g], Molmasse [g/mol], Volumen Lsgmittel [mL]
c_string = "C6H6, 1, 78, 100, C2H4O2, 0.8, 60, 95, C5H5N, 1.1, 79, 110"
c_list = c_string.split(",")

mol = c_list[::4]
mass = [float(el) for el in c_list[1::4]]
mol_mass = [float(el) for el in c_list[2::4]]
vol = [float(el) for el in c_list[3::4]]

c = [(mass[i] / mol_mass[i]) / vol[i] for i in range(len(vol))] # List
    ↳ comprehension
c_max = max(c)
for m, ci in zip(mol, c):
    if ci == c_max:
        print(f"Die Konzentration von {m} beträgt {ci} mol/L und ist am
    ↳ höchsten!")

```

40.0.3 Aufgabe: Finde den Punkt mit maximaler Intensität im Spektrum

```

[ ]: wavenumber = [1000, 1010, 1020, 1030, 1040, 1050, 1060, 1070, 1080, 1090, 1100,
    ↳ 1110, 1120, 1130, 1140, 1150, 1160, 1170, 1180, 1190, 1200]
intensity = [0.000, 0.034, 0.045, 0.052, 0.030, 0.020, 0.61, 0.75, 0.082, 0.
    ↳ 083, 0.069, 0.038, 0.010, 0.004, 0.030, 0.077, 0.091, 0.059, 0.090, 0.072, 0.
    ↳ 041]
# Finde das maximum!

print("Die maximale Intensität von {} wird bei einer Wellenzahl von {}
    ↳ erreicht")

```